



Formation Python

Récapitulatif des fonctionnalités enseignées

Ce document fait office de rappel. Si nécessité de précision sur les utilisations, cf Jupyter Notebook ou documentation python sur internet.



Table des matières

Session 1	4
Déclaration d'une variable	4
Modification d'une variable	4
Type de variable	4
Entiers / int / nombre 'rond'	4
Virgules flottantes / float / nombre à virgule.....	4
Chaine de caractères / string / texte.....	4
Booléen / boolean / valeur de vérité	4
Fonctions utiles	4
Type de variable	4
Modification du type	4
Demander valeur à l'humain	4
Afficher	4
Session 2	5
Interactions avec les chaines de caractères / string	5
Aller à la ligne	5
Méthodes pour les strings.....	5
Découpage et indexation d'un string	5
Opérations possibles	6
Opérateurs arithmétique et d'affectation	6
Opérateurs de comparaison.....	6
Session 3	7
Objets à stockage multiple	7
Listes	7
Tuple / n-uplets	8
Session 4	8
Dictionnaire	8
Bloc de condition	9
Session 5	9
Boucles	9
For : itérations connues.....	10
While : tant qu'on a pas remplie une condition.....	10
Session 6	10
Module random.....	10



Déclarer	10
Générer un nombre aléatoire:	10
Générer un entier aléatoire:	10
Choisir un élément aléatoire dans une liste:.....	11
Mélanger une liste aléatoirement:.....	11
Module time	11
Importer le module:	11
Obtenir l'heure actuelle:	11
Mesurer des intervalles de temps:.....	11



Session 1

Déclaration d'une variable

`a = 3`

Modification d'une variable

`a = 'nouvelle valeur'`

Type de variable

Entiers / int / nombre 'rond'

`x = 6`

Virgules flottantes / float / nombre à virgule

`x = 7.6`

Chaine de caractères / string / texte

`t = 'Bonjour'`

Booléen / boolean / valeur de vérité

`v = True`

`v = False`

Fonctions utiles

Type de variable

`type(object)` renvoie le type de la variable object

Modification du type

`str(object)` renvoie la variable object sous forme de string

`int(object)` renvoie la variable object sous forme de int

`float(object)` renvoie la variable object sous forme de float

Demander valeur à l'humain

`reponse = input('Question ?')` enregistre une valeur string renseignée par l'opérateur dans la variable reponse

Afficher

`print(object)`



Session 2

Interactions avec les chaînes de caractères / string

Aller à la ligne

- Triples guillemets

```
'''Phrase numéro 1
```

```
Phrase numéro 2'''
```

- `\n`

```
print('Phrase 1. \nPhrase 2')
```

Méthodes pour les strings

- Appartenance d'un caractère ou d'une chaîne à une autre chaîne

'c' in 'Patrick' renvoie True

'a' not in 'Patrick' renvoie True

- Longueur d'une chaîne de caractères

```
len(mychaîne)
```

- Manipuler la chaîne

`mychaîne.capitalize()` met la première lettre en majuscule

`mychaîne.strip()` supprime les espaces en début et fin

`mychaîne.upper()` met toutes les lettres en majuscule

`mychaîne.lower()` met toutes les lettres en minuscule

- Compter la présence d'un élément dans une chaîne

`mychaîne.count('o')` renvoie le nombre de fois que la lettre o apparaît dans la chaîne

- Localiser un élément dans une chaîne

`mychaîne.find('bout')` renvoie l'indice de la 1^{ère} lettre

Découpage et indexation d'un string

Rappel

V	O	I	L	A
0	1	2	3	4
-5	-4	-3	-2	-1



- Chercher un caractère par son index

mychaine[0] renvoie le 1^{er} caractère

mychaine[-1] renvoie le dernier caractère

mychaine[n] renvoie le énième caractère

- Récupérer une partie de la chaine

mychaine[0:7] ou mychaine[:7] renvoie les 7 premiers caractères

mychaine[7:] renvoie les derniers caractères à partir du 7^{ème}

mychaine[3:5] renvoie le 3^{ème} et 4^{ème} caractère *indice de fin exclus !!*

Opérations possibles

Opérateurs arithmétique et d'affectation

- Addition : + entre des int ou des float
- Concaténation : + entre des strings
- Soustraction : -
- Multiplication : * (aussi avec des strings)
- Division : /
- Modulo : %
- Exponentiation : **
- Division entière : //
- Valeur absolue : abs()

a += 1

a -= 1

a *= 1

a /= 1

Opérateurs de comparaison

- Égalité : ==
- Inégalité : !=
- Infériorité : <
- Supériorité : >
- Infériorité ou égalité : <=
- Supériorité ou égalité : >=

renvoie une valeur booléenne : True ou False



Session 3

Objets à stockage multiple

Listes

Peuvent contenir tout type de données

- Déclaration

```
nomliste = [element1, element2]
```

- Longueur de la liste

```
len(nomliste)
```

- Appeler un/plusieurs élément(s) de la liste avec son index n

```
nomliste[n]
```

```
nomliste[n:n+3]
```

- Accéder à l'index/la position d'un élément dans la liste

```
nomliste.index(element1)
```

- Présence d'un élément dans la liste

`element5 in nomliste` renvoie False

- Modification d'un élément de la liste

```
nomliste[0] = 'nouvelle valeur'
```

- Ajouter un élément

`nomliste.append(new_element)` rajoute à la fin de la liste

`nomliste.insert(index, new_element)` rajoute à une position précise dans la liste

`nomliste.extend(liste2)` ajoute plusieurs éléments à la fois

- Supprimer des éléments

```
nomliste.remove(element1)
```

```
nom.pop(index)
```

`nom.pop()` supprime le dernier élément

- Trier les éléments (ordre alphabétique ou numérique)

`nomliste.sort()` par ordre alphabétique ou numérique

`nomliste.reverse()` inverse l'ordre



- Dénombrer la présence d'un élément

```
nomliste.count(element)
```

Tuple / n-uplets

Peuvent contenir tout type de données, immuables ! (Une fois créé, impossible de modifier)

- Déclaration

```
montuple = (element1, element2)
```

- Accéder à un ou plusieurs élément(s) par son index n

```
montuple[0]
```

```
montuple[6:9]
```

- Obtenir l'index/la position d'un élément

```
montuple.index(element1)
```

- Longueur d'un tuple

```
len(montuple)
```

- Dénombrer les occurrences d'un élément dans un tuple

```
montuple.count(element)
```

- Vérifier la présence d'un élément dans le tuple

```
element in montuple
```

- Créer un tuple à partir d'un autre

```
tuple2 = montuple.__add__(tuple3)
```

```
tuple2 = montuple + tuple3
```

Session 4

Dictionnaire

Collections de paires clé-valeur

- Déclaration

```
mondico = { cle1 : valeur1, cle2 : valeur2 }
```

- Liste des paires clé-valeur

```
mondico.keys() renvoie les clés
```

```
mondico.values() renvoie les valeurs
```

```
mondico.items() renvoie les paires
```




- Accéder à une valeur

```
mondico[cle1]
```

```
mondico.get(cle1)
```

- Ajouter une valeur

```
mondico[new_cle] = new_valeur
```

- Modifier une valeur

```
mondico[cle1] = new_valeur
```

- Supprimer une valeur

```
del mondico[cle1]
```

```
mondico.pop(cle1)
```

- Mettre à jour un dictionnaire par un autre

```
dico.update(dico2)
```

Bloc de condition

Ne pas oublier les alinéas !!! ils déterminent ce qui fait partie du bloc à exécuter si la condition est remplie

if condition :

 bloc à exécuter

elif condition2 :

 bloc à exécuter

else :

 bloc à exécuter

elif et else sont optionnels

la condition du elif ne doit pas recouper la condition du if

ne jamais indiquer de condition au else, il comprend de base tous les cas opposés aux conditions indiquées dans les if et elif

Session 5

Boucles



For : itérations connues

for i in range() :

 bloc d'exécution

i étant une variable locale à la boucle, on ne peut pas l'appeler en dehors de la boucle

range(5) : permet d'aller de 0 à 4 *rappel indice de fin exclus*

for element in liste :

 bloc à exécuter

les boucles for peuvent parcourir des listes/tuples/dictionnaires/string

While : tant qu'on a pas remplie une condition

while condition :

 bloc à exécuter

la boucle sera exécutée tant que la condition est juste

Session 6

Module random

Random est un module intégré à Python qui permet de générer des nombres aléatoires. Il permet de faire appel à des fonctions spécifiques : random, randint, choice, shuffle

Déclarer

import random

Générer un nombre aléatoire:

La fonction principale du module random est random.random(). Elle renvoie un nombre flottant aléatoire compris entre 0 (inclus) et 1 (exclu).

nombre_aleatoire = random.random()

Générer un entier aléatoire:

Pour obtenir un entier aléatoire dans une plage spécifique, utilisez random.randint(a, b).



```
nombre_entier = random.randint(1, 10) # Génère un entier entre 1 et 10 (inclus)
```

Choisir un élément aléatoire dans une liste:

La fonction `random.choice(sequence)` permet de sélectionner un élément aléatoire dans une liste, une chaîne de caractères ou un autre type de séquence.

```
element_aleatoire = random.choice(liste)
```

Mélanger une liste aléatoirement:

La fonction `random.shuffle(sequence)` mélange les éléments d'une liste aléatoirement.

```
random.shuffle(liste)
```

Module time

Le module `time` est un module intégré à Python qui permet de travailler avec le temps. Il offre des fonctions pour obtenir l'heure actuelle, mesurer des intervalles de temps, mettre en pause l'exécution d'un programme, etc.

Importer le module:

```
import time
```

Obtenir l'heure actuelle:

`time.time()`: Renvoie le nombre de secondes écoulées depuis le 1er janvier 1970 (epoch) en tant que flottant.

`time.localtime()`: Renvoie une structure de données représentant l'heure locale actuelle.

`time.strftime(format, t)`: Formate une date et une heure selon un format spécifié.

Mesurer des intervalles de temps:

`time.sleep(seconds)`: Met en pause l'exécution du programme pendant un nombre spécifié de secondes.

Pour mesurer le temps d'exécution d'un bloc de code, vous pouvez utiliser `time.time()` avant et après le bloc et calculer la différence.